AMIA

INFORMATICS PROFESSIONALS. LEADING THE WAY.

2025 ANNUAL SYMPOSIUM

WHERE COLLABORATION MEETS HEALTHCARE TRANSFORMATION

#AMIA2025

AMIA KDDM Collaborative Workshop on Open-Source Large Language Models for Health Applications: Performance, Multimodality, and Real-World Implementations

November 15, Marriott Marquis, Atlanta

Hands-on Tutorial

Building LLM Applications with LangChain and LangGraph

Balu Bhasuran, PhD
Visiting Assistant Professor
School of Information
Florida State University

FSU | FLORIDA STATE UNIVERSITY

FSU | eHEALTH LAB
SCHOOL OF INFORMATION

DISCLOSURE OF RELEVANT FINANCAL RELATIONSHIPS

The following presenter **have no** relevant financial relationship(s) with ineligible companies to disclose.

- Balu Bhasuran, PhD

LEARNING OBJECTIVES

At the conclusion of this presentation the learner will be able to:

- **Implement** workflows using **LangChain** and **LangGraph** to operationalize open-source large language models (LLMs).
- **Configure and execute** API calls to interact with **locally hosted LLMs** for customized inference and reasoning tasks.
- **Utilize** the **‘Jan’ graphical interface** to efficiently access, manage, and deploy open-source LLMs in a user-friendly environment.

LangChain + LangGraph: API Access for Local LLMs

Key Capabilities

- Build modular LLM pipelines using **LangChain** and **LangGraph**
- Supports **multi-step workflows:** read → reason → respond
- Access **locally hosted LLMs** via API calls — no cloud dependency
- Integrate open-source models (e.g., LLaMA, Mistral, Gemma, Mixtral) seamlessly
- Enables **private, offline inference** — **no internet or external data sharing**

GPT 5

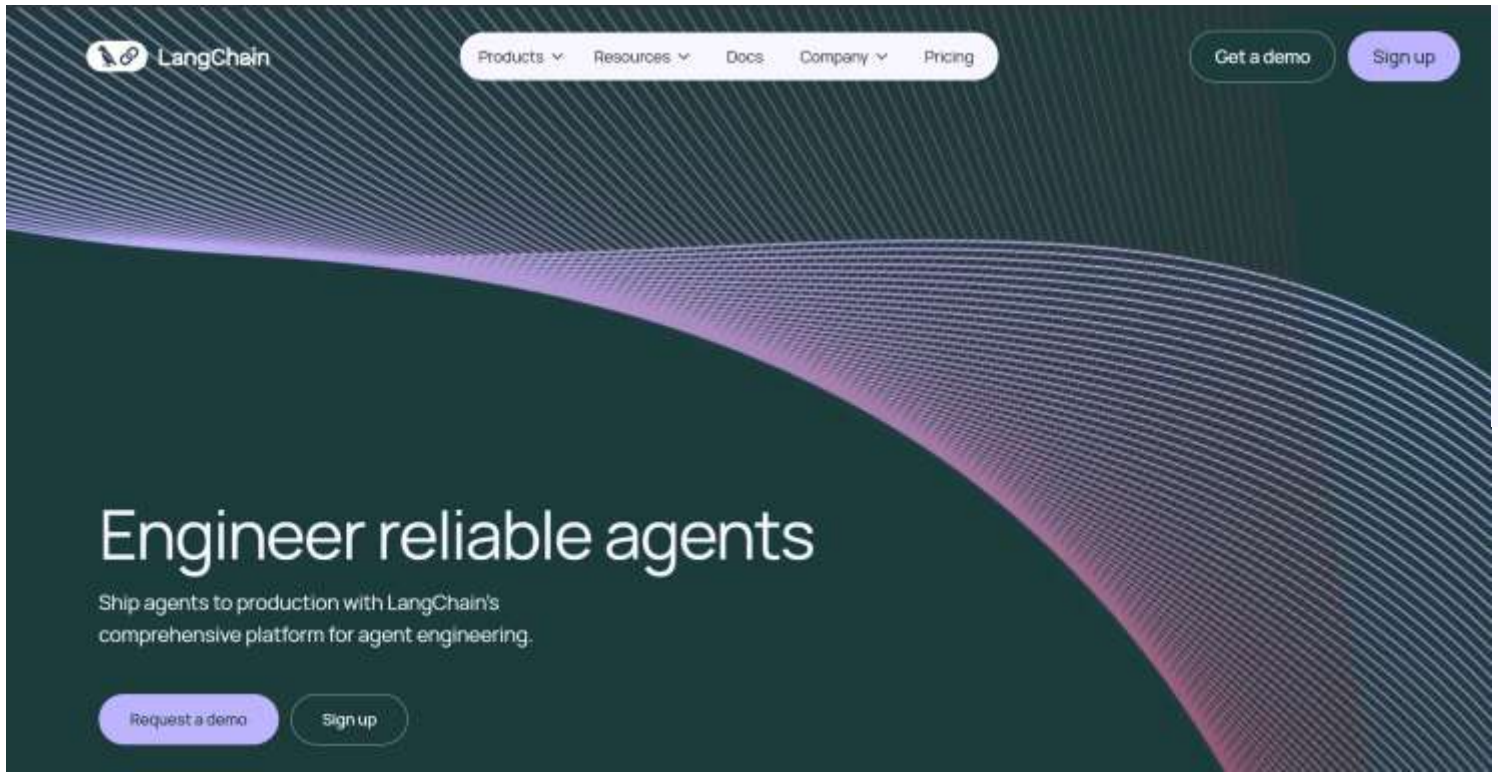
Advantages of Local API Access

1. **Privacy & Control**
   - Data never leaves your environment
   - Full control over computation and storage
   - Ideal for clinical, legal, and institutional data

2. **Flexibility & Integration**
   - Supports diverse data formats: **JSON**, **CSV**, **SQL**, **text**, **PDFs**
   - Works across multiple programming languages (Python, JavaScript, etc.)
   - Can be connected to external data sources or APIs for hybrid reasoning

3. **Efficiency & Scalability**
   - Run **hundreds of prompts locally** without latency or cost**LLaMA**, **Gemma**, **or Mixtral**
   - Rep ickly during experimentation
   - Cac to accelerate development cyc

LangChain



https://www.langchain.c

Jupyter Notebook

```
!pip install -U pydantic langgraph langchain langchain-core langchain-community ctransformers
```

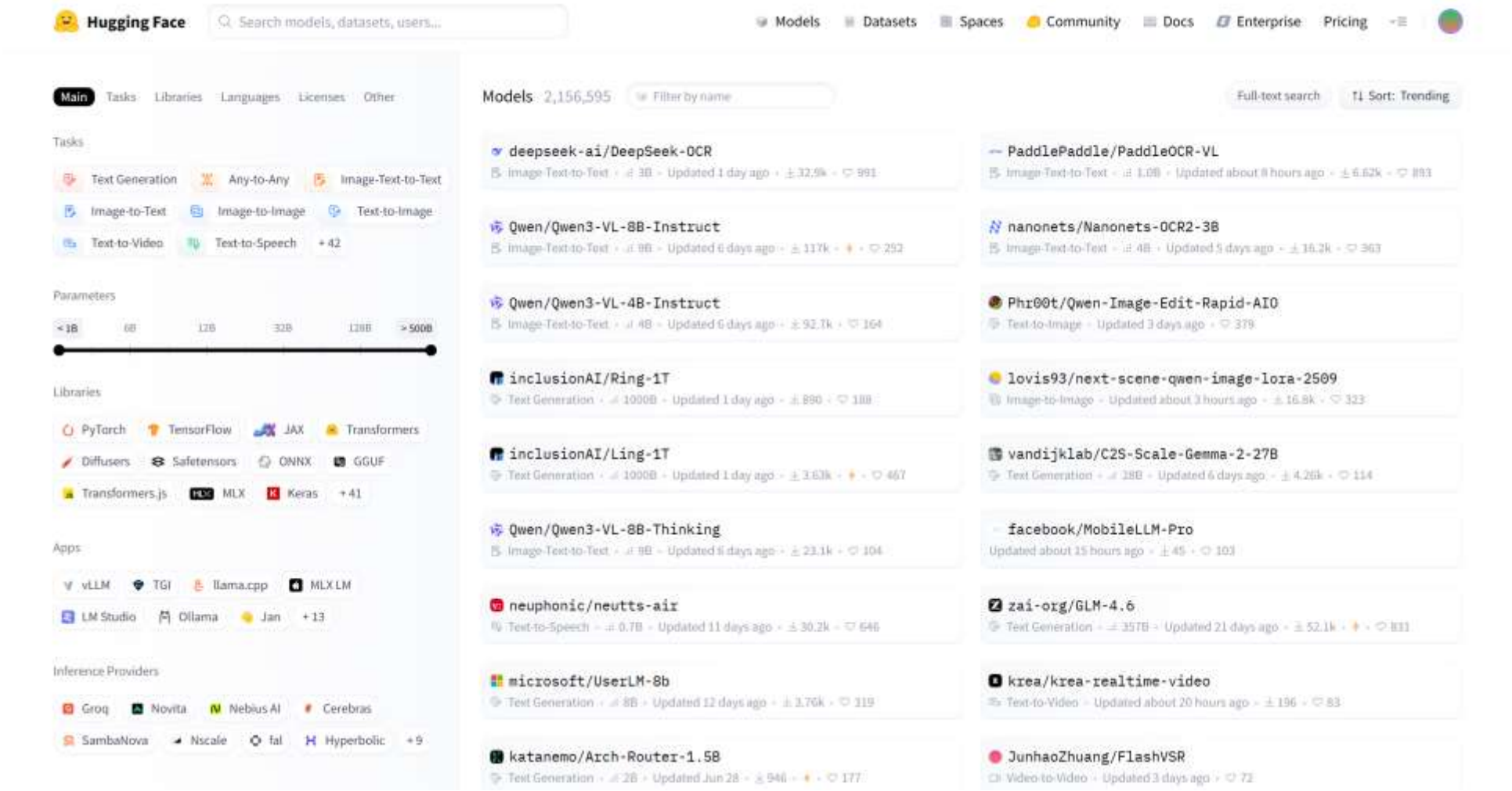
```
Requirement already satisfied: pydantic in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (2.12.3)
Requirement already satisfied: langgraph in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (1.0.1)
Requirement already satisfied: langchain in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (1.0.1)
Requirement already satisfied: langchain-core in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (1.0.0)
Requirement already satisfied: langchain-community in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (0.4)
Requirement already satisfied: ctransformers in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (0.2.27)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from pydantic) (2.41.4)
Requirement already satisfied: typing-extensions>=4.14.1 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from pydantic) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from pydantic) (0.4.2)
Requirement already satisfied: langgraph-checkpoint<4.0.0,>=2.1.0 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from langgrap
h) (3.0.0)
Requirement already satisfied: langgraph-prebuilt<1.1.0,>=1.0.0 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from langgraph)
(1.0.1)
Requirement already satisfied: langgraph-sdk<0.3.0,>=0.2.2 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from langgraph) (0.2.
9)
Requirement already satisfied: xxhash>=3.5.0 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from langgraph) (3.6.0)
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in c:\users\bb23u\appdata\local\anaconda3\lib\site-packages (from langchain-core) (1.
```

Jupyter Notebook

[2]:
```
from langchain_community.llms import CTransformers
```

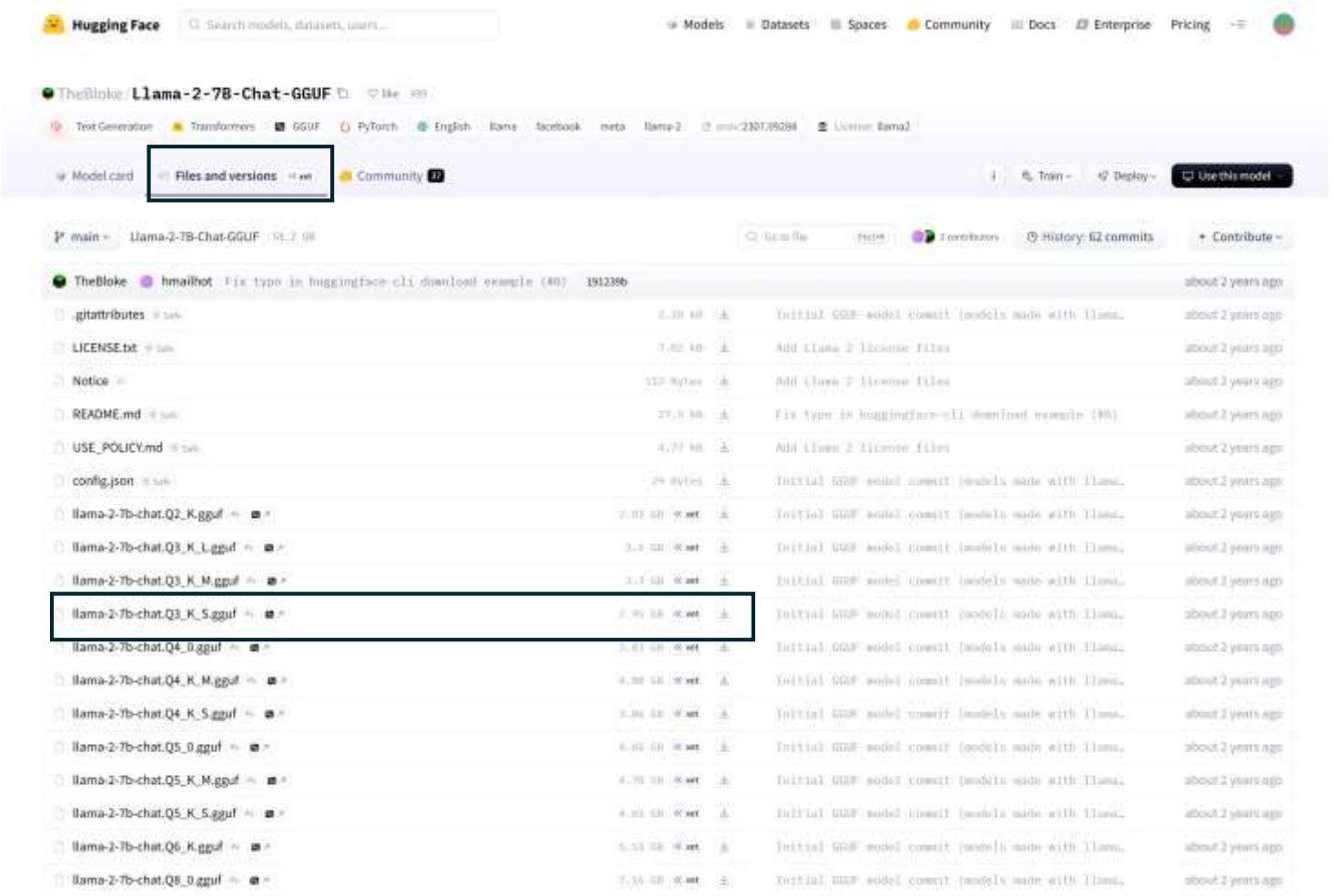
[3]:
```
from langchain_core.callbacks import AsyncCallbackManager
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain_community.llms import CTransformers

# Set up callback manager
callback_manager = AsyncCallbackManager([StreamingStdOutCallbackHandler()])
```

Open sourced LLMs



https://huggingface.co/model

Open sourced LLMs



https://huggingface.co/TheBloke/Llama-2-7B-Chat-GGUF/tree/main

Open source LLMs

```
!pip install huggingface_hub[hf_xet]
from huggingface_hub import hf_hub_download
import os

# Set your target directory
local_dir = r"D:\Open LLMs"

# Create the folder if it doesn't exist
os.makedirs(local_dir, exist_ok=True)

# Hugging Face repository and file name
repo_id = "TheBloke/Llama-2-7B-Chat-GGUF"
filename = "llama-2-7b-chat.Q3_K_S.gguf"

# Download the file to the specified local directory
local_path = hf_hub_download(
    repo_id=repo_id,
    filename=filename,
    repo_type="model",
    local_dir=local_dir
)

print(f"✅ File downloaded successfully!\nSaved at: {local_path}")
```

```
Downloading hf_xet-1.1.10-cp37-abi3-win_amd64.whl (2.8 MB)
   ---------------------------------------- 0.0/2.8 MB ? eta -:--:--
   ---------------- ---------------------- 1.2/2.8 MB 37.4 MB/s eta 0:00:01
   ---------------------------------------  2.8/2.8 MB 44.2 MB/s eta 0:00:01
   ---------------------------------------- 2.8/2.8 MB 35.4 MB/s eta 0:00:00
Installing collected packages: hf-xet
Successfully installed hf-xet-1.1.10
Error displaying widget
✅ File downloaded successfully!
Saved at: D:\Open LLMs\llama-2-7b-chat.Q3_K_M.gguf
```

https://huggingface.co/TheBloke/Llama-2-7B-Chat-GGUF/tree/main

```
# Model config
config = {
    'max_new_tokens': 2048,
    'repetition_penalty': 1.1,
    'temperature': 0.7,
    'top_k': 50
}

# Load local model
llm = CTransformers(
    model=r"D:\Open LLMs\llama-2-7b-chat.Q3_K_S.gguf",
    model_type="llama",
    config=config,
    callback_manager=callback_manager,
    verbose=True
)
```

[6]:
```
#Example 1
prompt = """
Highlight the most clinically important concepts in the following *Assessment and Plan* section by wrapping them in HTML <span> tags.
Focus on diagnoses, medications, procedures, and follow-up plans that are essential for clinical decision-making.

Input text:
Assessment and Plan:
Patient with known Coronary Artery Disease s/p CABG, presenting for post-op follow-up.
Blood pressure remains elevated despite current beta blocker and diuretic therapy.
Continue aspirin and statin.
Initiate ACE inhibitor for improved BP control.
Encourage dietary modifications and daily walking.
Schedule repeat lipid panel in 6 weeks and cardiology follow-up in 2 months.
"""
response = llm.invoke(prompt)
print("\n\nResponse:\n", response)
```

[6]:
```python
#Example 1
prompt = """
Highlight the most clinically important concepts in the following *Assessment and Plan* section by wrapping them in HTML <span> tags.
Focus on diagnoses, medications, procedures, and follow-up plans that are essential for clinical decision-making.

Input text:
Assessment and Plan:
Patient with known Coronary Artery Disease s/p CABG, presenting for post-op follow-up.
Blood pressure remains elevated despite current beta blocker and diuretic therapy.
Continue aspirin and statin.
Initiate ACE inhibitor for improved BP control.
Encourage dietary modifications and daily walking.
Schedule repeat lipid panel in 6 weeks and cardiology follow-up in 2 months.
"""
response = llm.invoke(prompt)
print("\n\nResponse:\n", response)
```

```
Response:

Output:
<span>Diagnoses:</span> Coronary Artery Disease, Hypertension

<span>Medications:</span> Aspirin, Statin, Beta Blocker, Diuretic, ACE Inhibitor

<span>Procedures:</span> CABG (past)

<span>Follow-up plans:</span> Repeat lipid panel in 6 weeks, Cardiology follow-up in 2 months

Note: The output is a summary of the most clinically important concepts in the input text.
```

[8]:
```
#Example 2
prompt = """
You are reviewing a physician's clinical note.
provide a concise summary (2-3 sentences) describing the patient's condition, key findings, treatment, and plan.

Input text:
Progress Note:
Mrs. Adams, a 72-year-old female with a history of chronic obstructive pulmonary disease (COPD) and hypertension,
presented with worsening cough and dyspnea.
O₂ saturation on arrival was 88%.
Chest X-ray revealed bilateral infiltrates consistent with pneumonia.
Started on IV antibiotics (ceftriaxone, azithromycin) and supplemental oxygen via nasal cannula.
Continue home antihypertensives.
Plan to reassess respiratory status and transition to oral antibiotics once stable.
Follow-up chest imaging in 2 weeks.
Please write a summary
"""

response = llm.invoke(prompt)
print("\n\nResponse:\n", response)
```

[8]:
```
#Example 2
prompt = """
You are reviewing a physician's clinical note.
provide a concise summary (2–3 sentences) describing the patient’s condition, key findings, treatment, and plan.

Input text:
Progress Note:
Mrs. Adams, a 72-year-old female with a history of chronic obstructive pulmonary disease (COPD) and hypertension,
presented with worsening cough and dyspnea.
O₂ saturation on arrival was 88%.
Chest X-ray revealed bilateral infiltrates consistent with pneumonia.
Started on IV antibiotics (ceftriaxone, azithromycin) and supplemental oxygen via nasal cannula.
Continue home antihypertensives.
Plan to reassess respiratory status and transition to oral antibiotics once stable.
Follow-up chest imaging in 2 weeks.
Please write a summary
"""

response = llm.invoke(prompt)
print("\n\nResponse:\n", response)
```

```
Response:
 Summary: Mrs. Adams, a 72-year-old female with COPD and hypertension, presented with worsening cough and dyspnea. Her oxygen saturation was 88% on arrival. A chest X-ray revealed bilateral infiltrates consistent with pneumonia, and she was started on IV antibiotics and supplemental oxygen via nasal cannula. She will continue to take her home antihypertensives and follow-up imaging is planned in 2 weeks.
```

Iteration over ‘n’ number of files

[10]:
```python
#Iteration over a JSON file
import json
# Load JSON notes
with open("D:\Open LLMs\clinical_notes.json", "r") as f:
    notes = json.load(f)
notes[0]
```

[10]:
```
{'patient_id': 'P001',
 'note': 'Mr. Thomas, a 68-year-old male with diabetes and hypertension, presented with dizziness and elevated blood sugar. Started on insulin glargine 10 units nightly and advised low-carb diet. Scheduled endocrinology follow-up in 1 month.'}
```

[11]:

```
# Define prompt template
prompt_template = """
Please do two things:
You are reviewing a physician's clinical note.
provide a concise summary (2-3 sentences) describing the patient's condition, key findings, treatment, and plan.

Input text:
{note}
"""
```

```python
# Iterate through all notes
results = []
for entry in notes:
    patient_id = entry["patient_id"]
    note_text = entry["note"]
    prompt = prompt_template.format(note=note_text)

    print(f"\n🩺 Processing Patient {patient_id}...\n")
    response = llm.invoke(prompt)

    results.append({
        "patient_id": patient_id,
        "input_note": note_text,
        "model_output": response
    })

# Save results to a new JSON file
with open("D:\Open LLMs\clinical_results.json", "w") as f:
    json.dump(results, f, indent=2)

print("\n✅ Processing complete! Results saved to 'clinical_results.json'.")
```

```python
# Iterate through all notes
results = []
for entry in notes:
    patient_id = entry["patient_id"]
    note_text = entry["note"]
    prompt = prompt_template.format(note=note_text)

    print(f"\n🩺 Processing Patient {patient_id}...\n")
    response = llm.invoke(prompt)

    results.append({
        "patient_id": patient_id,
        "input_note": note_text,
        "model_output": response
    })

# Save results to a new JSON file
with open("D:\Open LLMs\clinical_results.json", "w") as f:
    json.dump(results, f, indent=2)

print("\n✅ Processing complete! Results saved to 'clinical_results.json'.")
```

```
🩺 Processing Patient P001...

🩺 Processing Patient P002...

🩺 Processing Patient P003...

✅ Processing complete! Results saved to 'clinical_results.json'.
```

```
results[0]
```

```
{'patient_id': 'P001',
 'input_note': 'Mr. Thomas, a 68-year-old male with diabetes and hypertension, presented with dizziness and elevated blood sugar. Started on i
nsulin glargine 10 units nightly and advised low-carb diet. Scheduled endocrinology follow-up in 1 month.',
 'model_output': '\nYour response:\nMr. Thomas is a 68-year-old male with a history of diabetes and hypertension who presented to the clinic w
ith symptoms of dizziness and elevated blood sugar. His clinical note indicates that he was started on insulin glargine 10 units nightly and a
dvised to follow a low-carb diet. He is scheduled for an endocrinology follow-up appointment in one month to monitor his progress.'}
```

LangGraph

Single agent

LangGraph

pypi v1.0.1 | downloads/month 12M | open issues 134 | docs latest

Trusted by companies shaping the future of agents – including Klarna, Replit, Elastic, and more – LangGraph is a low-level orchestration framework for building, managing, and deploying long-running, stateful agents.

https://www.langchain.com/lan

Graph API concepts

Graphs

At its core, LangGraph models agent workflows as graphs. You define the behavior of your agents using three key components:

1. `State`: A shared data structure that represents the current snapshot of your application. It can be any data type, but is typically defined using a shared state schema.
2. `Nodes`: Functions that encode the logic of your agents. They receive the current state as input, perform some computation or side-effect, and return an updated state.
3. `Edges`: Functions that determine which `Node` to execute next based on the current state. They can be conditional branches or fixed transitions.

By composing `Nodes` and `Edges`, you can create complex, looping workflows that evolve the state over time. The real power, though, comes from how LangGraph manages that state. To emphasize: `Nodes` and `Edges` are nothing more than functions - they can contain an LLM or just good ol' code.

Core benefits

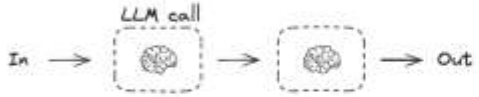
LangGraph provides low-level supporting infrastructure for *any* long-running, stateful workflow or agent. LangGraph does not abstract prompts or architecture, and provides the following central benefits:

- Durable execution: Build agents that persist through failures and can run for extended periods, automatically resuming from exactly where they left off.
- Human-in-the-loop: Seamlessly incorporate human oversight by inspecting and modifying agent state at any point during execution.
- Comprehensive memory: Create truly stateful agents with both short-term working memory for ongoing reasoning and long-term persistent memory across sessions.
- Debugging with LangSmith: Gain deep visibility into complex agent behavior with visualization tools that trace execution paths, capture state transitions, and provide detailed runtime metrics.
- Production-ready deployment: Deploy sophisticated agent systems confidently with scalable infrastructure designed to handle the unique challenges of stateful, long-running workflows.

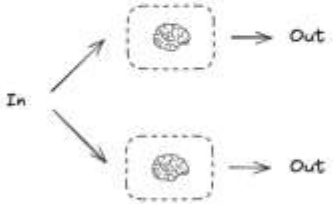
https://langchain-

Workflows

Prompt Chaining

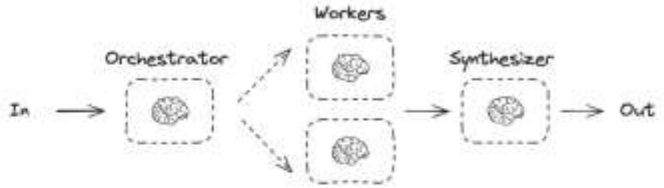


Parallelization

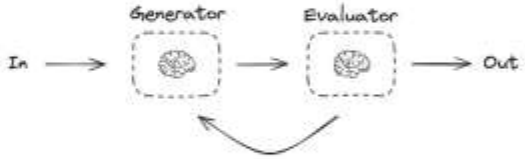


LLM is embedded in predefined code paths

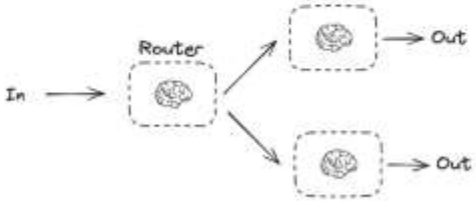
Orchestrator-Worker



Evaluator-optimizer

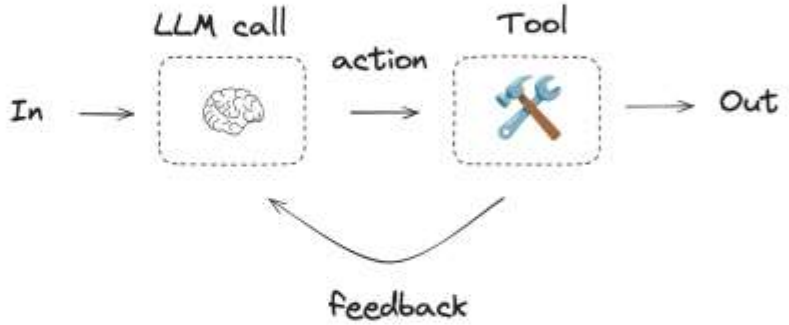


Routing



LLM directs control flow through predefined code paths

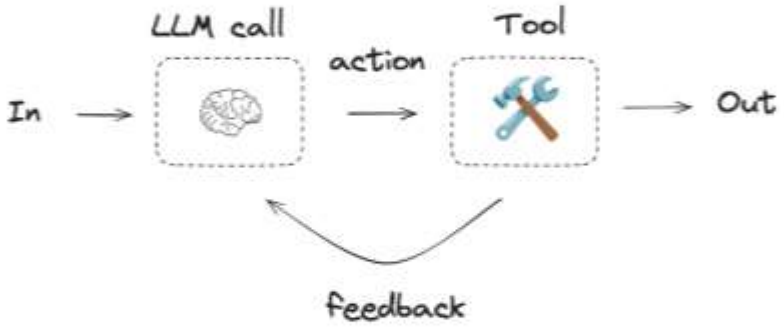
Agent



LLM directs its own actions based on environmental feedback

https://langchain-

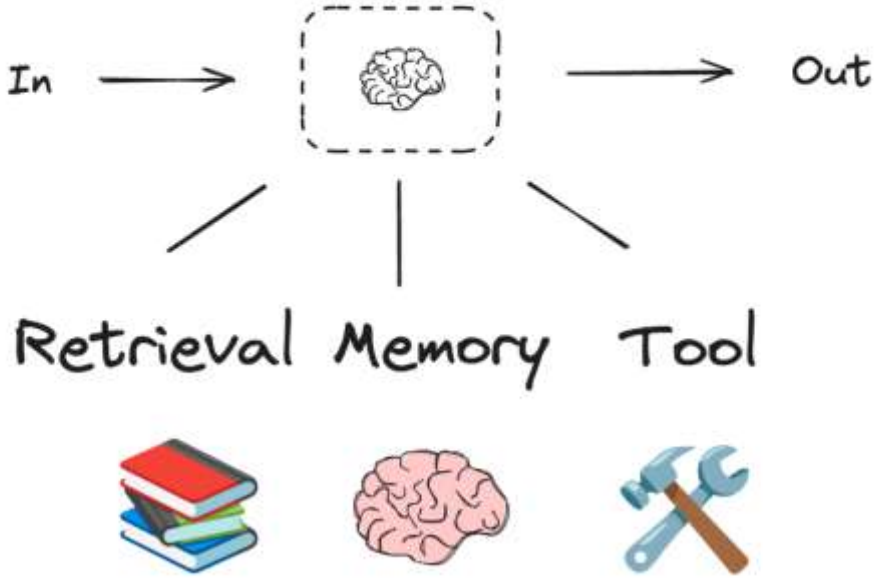
Agent



LLM directs its own actions based on environmental feedback

Building Blocks: The Augmented LLM

LLM have augmentations that support building workflows and agents. These include structured outputs and tool calling, as shown in this image from the Anthropic blog on `Building Effective Agents:`



https://langchain-

```python
from langgraph.graph import StateGraph, START, END
from langchain_community.llms import CTransformers
from langchain_core.messages import HumanMessage
import json
```

```
# ----------------------------
# Step 1: Define your model
# ----------------------------
llm = CTransformers(
    model=r"D:\Open LLMs\orca_mini_v2_13b.ggmlv3.q2_K.bin",
    model_type="llama",
    config={"max_new_tokens": 512, "temperature": 0.7},
    callback_manager=AsyncCallbackManager([StreamingStdOutCallbackHandler()]),
    verbose=False
)
```

```
prompt_template = """
Please do two things:
1. Wrap important medical entities (diagnoses, findings, medications, and follow-up plans) in HTML <span> tags.
2. Provide a concise summary (2–3 sentences) of the note.

Input text:
{note}
"""
```

```python
# ----------------------------
# Step 2: Define state structure
# ----------------------------
class ClinicalState(TypedDict, total=False):
    patient_id: str
    note: str
    prompt: str
    output: str
```

```python
# ----------------------------
# Step 3: Define graph nodes
# ----------------------------
def read_note(state: ClinicalState):
    """Create prompt text from the note."""
    formatted_prompt = prompt_template.format(note=state["note"])
    new_state = {**state, "prompt": formatted_prompt}
    return new_state

def run_llm(state: ClinicalState):
    """Run LLM on the prompt."""
    prompt = state.get("prompt", None)
    if not prompt:
        raise ValueError("Missing 'prompt' in state. Ensure read_note() ran first.")
    response = llm.invoke(prompt)
    return {**state, "output": response}

def show_output(state: ClinicalState):
    """Display the output neatly."""
    print(f"\n🩺 Patient {state['patient_id']} Result:")
    print(state["output"])
    return state
```

```python
# ----------------------------
# Step 4: Build LangGraph
# ----------------------------
graph = StateGraph(ClinicalState)

graph.add_node("read_note", read_note)
graph.add_node("run_llm", run_llm)
graph.add_node("show_output", show_output)

graph.add_edge(START, "read_note")
graph.add_edge("read_note", "run_llm")
graph.add_edge("run_llm", "show_output")
graph.add_edge("show_output", END)

app = graph.compile()
print(app.get_graph().draw_mermaid())
```

```python
# ----------------------------
# Step 4: Build LangGraph
# ----------------------------
graph = StateGraph(ClinicalState)

graph.add_node("read_note", read_note)
graph.add_node("run_llm", run_llm)
graph.add_node("show_output", show_output)

graph.add_edge(START, "read_note")
graph.add_edge("read_note", "run_llm")
graph.add_edge("run_llm", "show_output")
graph.add_edge("show_output", END)

app = graph.compile()
print(app.get_graph().draw_mermaid())
```

```
---
config:
  flowchart:
    curve: linear
---
graph TD;
        __start__([<p>__start__</p>]):::first
        read_note(read_note)
        run_llm(run_llm)
        show_output(show_output)
        __end__([<p>__end__</p>]):::last
        __start__ --> read_note;
        read_note --> run_llm;
        run_llm --> show_output;
        show_output --> __end__;
        classDef default fill:#f2f0ff,line-height:1.2
        classDef first fill-opacity:0
        classDef last fill:#bfb6fc
```

```python
with open("D:\Open LLMs\clinical_notes.json", "r") as f:
    notes = json.load(f)
notes[0]
```

```
{'patient_id': 'P001',
 'note': 'Mr. Thomas, a 68-year-old male with diabetes and hypertension, presented with dizziness and elevated blood sugar. Started on insulin glargine 10 units nightly and advised low-carb diet. Scheduled endocrinology follow-up in 1 month.'}
```

```python
for entry in notes:
    initial_state = {"patient_id": entry["patient_id"], "note": entry["note"], "output": ""}
    app.invoke(initial_state)
```

```
🩺 Patient P001 Result:

Output:
<span>Mr. Thomas, a 68-year-old male with diabetes and hypertension, presented with dizziness and elevated blood sugar.</span> <span>Started o
n insulin glargine 10 units nightly and advised low-carb diet.</span> <span>Scheduled endocrinology follow-up in 1 month.</span>

🩺 Patient P002 Result:

Output:
Ms. Rivera, a 45-year-old female with asthma, was experiencing increased wheezing and cough. She received nebulized albuterol and oral prednis
one from her physician. The plan is to taper steroids and continue taking an inhaled corticosteroid. Follow-up with the pulmonology clinic in
two weeks.

🩺 Patient P003 Result:

Output:
<span class="important">Mr. Patel, a 59-year-old male post coronary stent placement, presented for routine follow-up. His blood pressure is we
ll-controlled with aspirin and atorvastatin. The provider encouraged cardiac rehab and lifestyle modification.</span>


To summarize, the note describes a follow-up visit for a 59-year-old male who had a coronary stent placement. The patient is taking aspirin an
d atorvastatin and has well-controlled blood pressure. The provider encourages cardiac rehab and lifestyle modification.
```

Final Take-Aways

- **Implement** workflows using **LangChain** and **LangGraph** to operationalize open-source large language models (LLMs).
- **Configure and execute** API calls to interact with **locally hosted LLMs** for customized inference and reasoning tasks.
- **Utilize** the **‘Jan’ graphical interface** to efficiently access, manage, and deploy open-source LLMs in a user-friendly environment.

Questions?

AMIA

INFORMATICS PROFESSIONALS. LEADING THE WAY.